

EXPERIMENT -4

Student Name: Tejas Bhat

UID: 24BAI70851

Branch: AIT-CSE (AIML)

Section/Group: 24AIT_KRG-1A

Semester: 5

Subject Name: Database Management System

Subject Code: 24CSH-298

Q1)

- (i) You are given a table `tbl_happiness` that contains the ranking of happiness for different countries. Your task is to retrieve all the records from the table **while ensuring that "India" and "Sri Lanka" appear at the top, followed by the rest of the countries in their original ranking order.**

(B)

- (ii) Make all the rankings in descending order except **India** and **Sri Lanka**.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE) pgAdmin

(for PostgreSQL)

CODE SCREENSHOTS:

i)

EXPERIMENT -4

```
select *,
case
when country = 'India' then 1
when country = 'Sri Lanka' then 2
else 3
end as rank
from tbl_happiness
order by rank asc, rankings asc
```

ii)

```
select *,
case
when country = 'India' then 1
when country = 'Sri Lanka' then 2
else 3
end as rank
from tbl_happiness
order by rank asc, rankings desc
```

OUTPUT SCREENSHOTS:

i)

	sno [PK] integer	rankings integer	country character varying (50)	rank integer
1	8	126	India	1
2	9	128	Sri Lanka	2
3	1	1	Finland	3
4	2	2	Denmark	3
5	3	3	Iceland	3
6	4	4	Israel	3
7	5	5	Netherlands	3
8	6	6	Sweden	3
9	7	7	Norway	3

ii)

EXPERIMENT -4

	sno [PK] integer	rankings integer	country character varying (50)	rank integer
1	8	126	India	1
2	9	128	Sri Lanka	2
3	7	7	Norway	3
4	6	6	Sweden	3
5	5	5	Netherlands	3
6	4	4	Israel	3
7	3	3	Iceland	3
8	2	2	Denmark	3
9	1	1	Finland	3

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Learned how to implement custom sorting using the ORDER BY clause with conditional logic.
- Understood how to prioritize specific records while maintaining the required order for others.
- Learned to sort data in both ascending and descending order based on conditions.
- Gained experience in creating customized query outputs to meet reporting requirements.

CONCLUSION:

This experiment demonstrated how to customize the ordering of query results using SQL. By applying conditional sorting, "India" and "Sri Lanka" were displayed at the top, while the remaining countries were arranged according to the required ranking order, including descending order in the second task. This illustrates the use of SQL sorting techniques to meet specific reporting requirements.

EXPERIMENT -4

Q2)

A company maintains employee information in a single table where each employee may report to another employee acting as their manager. Prepare a report that displays the details of every employee who supervises at least one other employee. The report should include the manager's employee ID, name, total number of direct reports, and the average age of those reporting employees. The average age should be rounded to the nearest whole number and the final result should be sorted by employee ID.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE) pgAdmin
(for PostgreSQL)

CODE SCREENSHOTS:

EXPERIMENT -4

```
select e.employee_id, e.name,  
count(e.employee_id) as reports_count,  
round(avg(e2.age)) from employees as e  
join employees as e2  
on e2.reports_to=e.employee_id  
group by e.employee_id
```

OUTPUT SCREENSHOTS:

	employee_id [PK] integer	name character varying (50)	reports_count bigint	age integer	round numeric
1	9	Hercy	2	43	39

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Learned how to perform a self-join to relate employees with their managers within the same table.
- Understood how to count the number of direct reports using the COUNT() aggregate function.
- Learned to calculate the average age of reporting employees using the AVG() function and round the result.
- Gained experience in generating hierarchical reports and sorting the output using the ORDER BY clause.

EXPERIMENT -4

CONCLUSION:

This experiment demonstrated how to use a self-join to analyze hierarchical employeemanager relationships. By applying aggregate functions such as COUNT() and AVG(), the report successfully identified managers, calculated the number of direct reports, determined the average age of their reporting employees, and presented the results in ascending order of employee ID.